Universidad Galileo	Guatemala 12 de Octubre del 2026
Facultad: FISICC	**Alumno: Roberto Molina**
Curso: Microprocesadores	**Carnet: 24000849**
Sección: A	**Hora de Laboratorio: 1:00 a 1:50**
Auxiliar: Brandon Mendoza	**Día de Laboratorio: Lunes**

Laboratorio #8 ADC

Resumen:

En esta práctica se trabajó con el convertidor analógico-digital (ADC) del microcontrolador STM32G431 de la tarjeta NUCLEO-G431KB, esta vez programando todo en lenguaje ensamblador. La idea era leer el voltaje de un potenciómetro y usar ese valor para controlar cuatro LEDs, de forma que se fueran encendiendo uno por uno conforme se giraba la perilla.

El potenciómetro se conectó como divisor de voltaje entre 3.3 V y GND, con el pin del centro en PA1, que corresponde al canal 2 del ADC2. Los LEDs se conectaron en PA0, PA4, PA7 y PA2. Para que el ADC funcionara hubo que darle reloj desde el registro RCC_AHB2ENR, poner PA1 en modo analógico, elegir la resolución y el canal en los registros CFGR y SQR1, y seguir la secuencia de encendido en el registro CR: sacarlo del modo deep-power-down, activar su regulador de voltaje y habilitarlo.

En el ciclo principal se inicia una conversión, se espera a que termine y el resultado, que va de 0 a 1023, se compara contra cuatro valores que dividen el recorrido del potenciómetro en quintos. Según el rango en el que cae la lectura se encienden ninguno, uno, dos, tres o los cuatro LEDs. Al final se comentó la línea que escribe en el ODR para comprobar que, aunque el ADC siguiera convirtiendo, los LEDs ya no cambiaban al mover el potenciómetro.

Objetivos:

- Configurar el ADC del STM32G431 en lenguaje ensamblador, siguiendo su secuencia de encendido y trabajando directamente con la dirección de cada registro.
- Leer el voltaje de un potenciómetro y convertirlo en un número digital de 10 bits.
- Usar el valor leído para encender los LEDs según el recorrido del potenciómetro, dividiendo el rango de lectura en cinco partes iguales.

Teoría:

Tarjeta NUCLEO-G431KB

La NUCLEO-G431KB trae integrado el depurador/programador STLINK-V3E, que se usa para cargar y depurar el programa sin necesidad de un programador externo. Además, el STLINK-V3E funciona como un puerto serial virtual por USB, y para eso se conecta al microcontrolador por PA2 (TX) y PA3 (RX) [1].

Convertidor analógico-digital (ADC)

Un ADC toma un voltaje que puede tener cualquier valor dentro de un rango y lo convierte en un número que el microcontrolador puede procesar. Ese número no es continuo, sino que depende de cuántos bits tenga la conversión: con n bits hay 2^n niveles posibles, así que con 10 bits la lectura va de 0 a 1023 y con 12 bits de 0 a 4095 [2].

El tamaño de cada escalón, llamado LSB, se obtiene dividiendo el voltaje de referencia entre la cantidad de niveles:

$$LSB = \frac{V_{ref}}{2^n}$$

En la tarjeta la referencia es de 3.3 V, entonces con 10 bits cada escalón vale:

$$LSB = \frac{3.3\ V}{1024} \approx 3.2\ mV$$

Cualquier cambio de voltaje menor a eso no alcanza a notarse en la lectura.

ADC de aproximaciones sucesivas

El STM32G431 usa convertidores de aproximaciones sucesivas, también llamados SAR [3]. Este tipo de ADC primero “congela” el voltaje de entrada con un circuito de muestreo y retención, y luego lo compara contra el voltaje de un DAC interno. Empieza probando el bit más significativo con el DAC a media escala: si la entrada es mayor, ese bit se queda en 1, y si es menor, se pone en 0. Después repite lo mismo con el siguiente bit, y así hasta llegar al menos significativo [4]. Por eso la conversión toma un paso por cada bit, y bajar la resolución la hace más rápida.

ADC del STM32G431

Según su hoja de datos, el STM32G431 tiene dos ADC con resolución nativa de 12 bits, que pueden llegar a 4 millones de muestras por segundo y aceptan voltajes de 0 a 3.6 V [3]. Las entradas están multiplexadas, es decir, cada ADC tiene varios canales y en el registro de secuencia se escoge cuál se va a leer. La resolución también se puede bajar a 10, 8 o 6 bits para convertir más rápido [3]. En la tabla de pines del datasheet aparece que PA0 es ADC12_IN1 y PA1 es ADC12_IN2 [3]. Esto quiere decir que PA1 corresponde al canal 2, y que lo pueden leer tanto el ADC1 como el ADC2.

Registros del ADC

- **RCC_AHB2ENR:** el bit 13 (ADC12EN) habilita el reloj del ADC1 y del ADC2, y el bit 0 el del puerto A [2].
- **GPIOx_MODER:** con el valor 11 en los dos bits de un pin, este queda en modo analógico, que es el que necesita el ADC [2].
- **ADC_CFGR:** en los bits 3 y 4 (RES) se elige la resolución (00 = 12 bits, 01 = 10 bits) y el bit 12 (OVRMOD) indica que, si llega un dato nuevo sin haber leído el anterior, este se sobrescribe [2].
- **ADC_SQR1:** en los bits 0 a 3 (L) se indica cuántas conversiones tiene la secuencia, menos uno, y en los bits 6 a 10 (SQ1) el canal de la primera conversión [2].
- **ADC_CCR:** es un registro compartido por el ADC1 y el ADC2. Con los bits 16 y 17 (CKMODE) en 11, el ADC usa el reloj del bus AHB dividido entre 4 [2].
- **ADC_CR:** aquí están DEEPPWD (bit 29) para salir del modo de bajo consumo, ADVREGEN (bit 28) para encender el regulador interno, ADEN (bit 0) para habilitar el ADC y ADSTART (bit 2) para iniciar una conversión [2].
- **ADC_ISR:** tiene las banderas ADRDY (bit 0), que avisa que el ADC ya está listo, y EOC (bit 2), que avisa que terminó una conversión [2].
- **ADC_DR:** guarda el resultado de la última conversión. Al leerlo, la bandera EOC se limpia sola [2].

Secuencia de encendido del ADC

Después de un reset el ADC está en modo deep-power-down para ahorrar energía, así que no basta con poner ADEN en 1. Primero hay que limpiar DEEPPWD, luego activar el regulador de voltaje con ADVREGEN y esperar un tiempo corto a que se estabilice. Hasta entonces se puede poner ADEN en 1, y antes de pedir la primera conversión hay que esperar a que el hardware levante la bandera ADRDY [2].

Potenciómetro como divisor de voltaje

Un potenciómetro tiene tres terminales: los dos extremos de la pista resistiva y el cursor, que es el pin del centro. Si los extremos se conectan a la alimentación y a tierra, el voltaje del cursor va cambiando entre 0 V y el voltaje de alimentación conforme se gira la perilla, y ese pin se puede llevar directo a una entrada analógica [6].

En los potenciómetros lineales, que se marcan con la letra B, la resistencia cambia en proporción al giro, de modo que a la mitad del recorrido se tiene la mitad de la resistencia. En los logarítmicos, marcados con A, el cambio es lento al principio y rápido al final, por eso se usan más en controles de volumen [6]. En esta práctica se usó uno lineal de 10 kΩ (B10K).

Lenguaje ensamblador ARM

El Cortex-M4 trabaja con el set de instrucciones Thumb-2, que combina instrucciones de 16 y 32 bits [5]. En ensamblador no existen estructuras como GPIOA->ODR, así que a cada registro se le dio un nombre con la directiva EQU y se trabajó con pocas instrucciones [5]:

- **LDR y STR:** leen un valor de memoria hacia un registro del CPU y lo escriben de regreso. Con LDR r0, =NOMBRE se carga en r0 la dirección del registro.
- **ORR y BIC:** ponen bits en 1 o en 0 sin tocar los demás, igual que |= y &= ~ en C.
- **TST y CMP:** hacen un AND o una resta solo para actualizar las banderas del procesador, sin guardar el resultado. Después se usan saltos condicionales como BEQ, BNE o BLO.
- **B, BL, PUSH y POP:** B salta a una etiqueta y BL salta a una subrutina guardando la dirección de regreso en LR. Con PUSH {lr} al inicio y POP {pc} al final, cada subrutina regresa al punto desde donde se llamó.

Resultados prácticos:

Lab8_ADC > Source > main.s

```
1       GLOBAL __main
2       AREA main, CODE, READONLY
3       EXPORT __main
4       EXPORT __use_two_region_memory
5   __use_two_region_memory EQU 0
6
7   RCC_AHB2ENR EQU 0x4002104C
8   GPIOA_MODER EQU 0x48000000
9   GPIOA_ODR EQU 0x48000014
10  ADC2_ISR EQU 0x50000100
11  ADC2_CR EQU 0x50000108
12  ADC2_CFGR EQU 0x5000010C
13  ADC2_SQR1 EQU 0x50000130
14  ADC2_DR EQU 0x50000140
15  ADC12_CCR EQU 0x50000308
16
17
18
19  __main
20      bl adc_config
21      bl led_config
22  loop
23      bl adc_read
24      bl led_write
25      b  loop
```

Fig. 1. Definición de registros y ciclo principal

```
adc_config
    push {lr}
    ;Senales de reloj para ADC y GPIOA
    ldr r0, =RCC_AHB2ENR
    ldr r1,[r0]
    orr r1,#(1<<0) ; Habilitar reloj GPIOA
    orr r1,#(1<<13) ; Habilitar reloj ADC
    str r1,[r0]

    ;PA1 como entrada analógica
    ldr r0, =GPIOA_MODER
    ldr r1,[r0]
    orr r1,#(3<<2) ;PA1 como entrada analogica
    str r1,[r0]

    ;Overrun y resolucion de 10 bits
    ldr r0, =ADC2_CFGR
    ldr r1,[r0]
    orr r1,#(1<<12)
    orr r1,#(1<<3) ; RES=01
    str r1,[r0]

    ; Conversion en el canal 2
    ldr r0, =ADC2_SQR1
    ldr r1,[r0]
    orr r1,#(2<<6) ; canal 2, L=0, SQ1=2
    str r1,[r0]

    ; reloj del ADC = HCLK/4
    ldr r0,=ADC12_CCR
    ldr r1,[r0]
    orr r1,#(3<<16) ; CKMODE=11
    str r1,[r0]
```

Fig. 2. Configuración de relojes, PA1 y parámetros del ADC

```
61      ; salir del deep power mode
62      ldr r0,=ADC2_CR
63      ldr r1,[r0]
64      bic r1,#(1<<29) ; DEEPWD =0
65      str r1,[r0]
66
67      ; habilitar el regulador de voltaje
68      ldr r1,[r0]
69      orr r1,#(1<<28) ; ADVREGEN=1
70      str r1,[r0]
71
72      ;esperar a que el regulador se estabilice
73      ldr r2,=2000
74  adc_delay
75      subs r2,#1
76      bne adc_delay
77
78      ; habilitar el ADC
79      ldr r1,[r0]
80      orr r1,#(1<<0) ; ADEN=1
81      str r1,[r0]
82
83      ;esperar a que el ADC este listo
84      ldr r0,=ADC2_ISR
85  adc_wait_ready
86      ldr r1,[r0]
87      tst r1,#(1<<0) ; ADRDY=1?
88      beq adc_wait_ready
89
90      pop {pc}
91
```

Fig. 3. Secuencia de encendido del ADC

```
led_config
    push {lr}
    ldr r0,=GPIOA_MODER
    ldr r1,[r0]
    bic r1,#(3<<4) ; limpiar PA2
    orr r1,#(1<<4) ; PA2 como salida
    bic r1,#(3<<0) ; limpiar PA0
    orr r1,#(1<<0) ; PA0 como salida
    bic r1,#(3<<8) ; limpiar PA4
    orr r1,#(1<<8) ; PA4 como salida
    bic r1,#(3<<14) ; limpiar PA7
    orr r1,#(1<<14) ; PA7 como salida
    str r1,[r0]

    pop {pc}
```

Fig. 4. Configuración de los LEDs como salida

```
adc_read
    push {lr}
    ldr r1,=ADC2_CR
    ldr r2,[r1]
    orr r2,#(1<<2) ; ADSTART=1
    str r2,[r1]

    ;esperar a que la conversion termine
    ldr r1,=ADC2_ISR
adc_wait_eoc
    ldr r2,[r1]
    tst r2,#(1<<2) ; EOC=1?
    beq adc_wait_eoc

    ;leer el resultado
    ldr r1, =ADC2_DR
    ldr r0,[r1] ;se devuelve el valor de la conversion en r0
    pop {pc}

```

Fig. 5. Lectura de una conversión

```
127 led_write
128     push {lr}
129     mov r2,#0 ; r2 = LEDs a encender empieza en 0
130
131     ldr r3,=205
132     cmp r0,r3 ; comparar el valor de la conversion con 1/5 del maximo (1023)
133     blo led_output ; menos de 1/5 ningun LED
134     orr r2,#(1<<0) ; LED4 (PA0)
135
136     ldr r3, =409
137     cmp r0,r3
138     blo led_output ; menos de 2/5
139     orr r2,#(1<<4) ; LED3 (PA4)
140
141     ldr r3, =614
142     cmp r0,r3
143     blo led_output ; menos de 3/5
144     orr r2,#(1<<7) ; LED2 (PA7)
145
146     ldr r3, =818
147     cmp r0,r3
148     blo led_output ; menos de 4/5
149     orr r2,#(1<<2) ; LED1 (PA2)
150 led_output
151     ldr r1,=GPIOA_ODR
152     ldr r3,[r1]
153     bic r3,#0x95 ; limpiar bits de los LEDs
154     orr r3,r2 ; encender los LEDs correspondientes
155     str r3,[r1]
156     pop {pc}
```

Fig. 6. Control de los LEDs según la lectura del ADC

```
150 led_output
151     ldr r1,=GPIOA_ODR
152     ldr r3,[r1]
153     bic r3,#0x95 ; limpiar bits de los LEDs
154     orr r3,r2 ; encender los LEDs correspondientes
155     ;str r3,[r1]
156     pop {pc}
157
158
159     end
```

Fig. 7. Modificación del paso 5

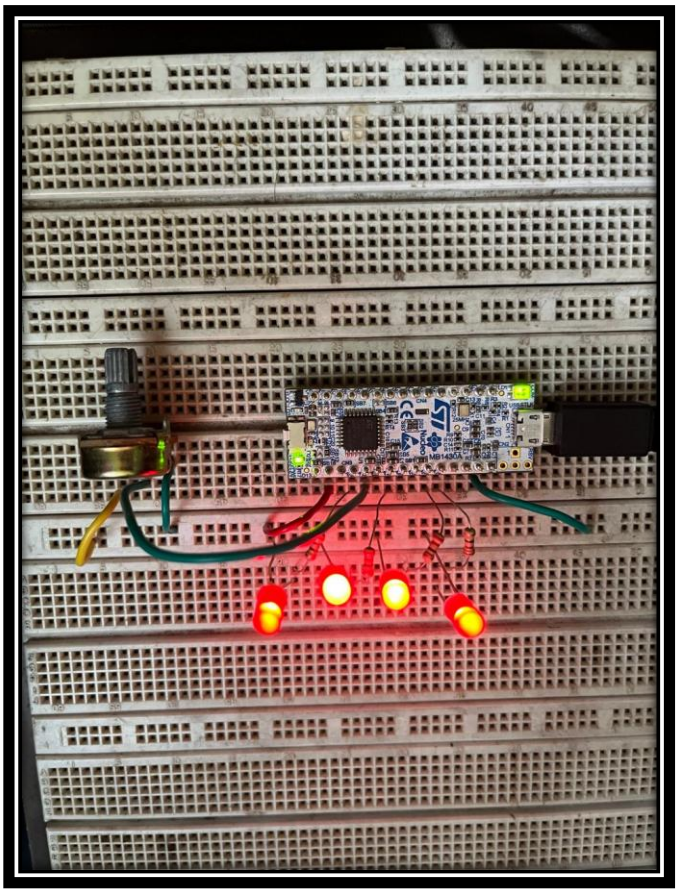


Fig. 8. Circuito

Video demostrativo: https://youtube.com/shorts/YLVtmBURXi8?si=V7FunPL_sICkbwZN

Procedimientos teóricos:

Definición de registros y ciclo principal (Fig. 1)

Al inicio se exportó la etiqueta __main, que es a donde salta el startup al arrancar, y con EQU se le asignó un nombre a la dirección de cada registro. Cada dirección es la dirección base del periférico más el offset del registro según el manual de referencia. Por ejemplo, el RCC empieza en 0x40021000 y AHB2ENR tiene un offset de 0x4C, por eso RCC_AHB2ENR queda en 0x4002104C. De la misma forma, el ADC2 empieza en 0x50000100 y el registro común CCR queda en 0x50000308.

El programa principal llama una sola vez a las subrutinas de configuración y después se queda en un ciclo infinito que lee el ADC y actualiza los LEDs.

Relojes, pin analógico y parámetros del ADC (Fig. 2)

Todas las configuraciones siguen el mismo patrón: se carga la dirección del registro en r0, se lee su valor con LDR, se cambian solo los bits necesarios con ORR o BIC y se vuelve a escribir con STR. Así no se borran por error otros bits del registro.

Primero se encendieron los bits 0 y 13 de RCC_AHB2ENR para darle reloj al puerto A y al ADC. Después se escribió 11 en los bits 2 y 3 de GPIOA_MODER para dejar PA1 en modo analógico. En CFGR se activó OVRMOD (bit 12) y se puso RES en 01 para trabajar con 10 bits, igual que en el código de ejemplo de la guía. En SQR1 se dejó L en 0, o sea una sola conversión, y en SQ1 se escribió el canal 2 desplazado 6 posiciones. Por último, en el registro común CCR se puso CKMODE en 11 para que el ADC use el reloj del AHB dividido entre 4.

Corrección del canal

El código en C del anexo pone PA1 en modo analógico, pero en SQR1 selecciona el canal 1. Al revisar la tabla de pines del datasheet se vio que el canal 1 es PA0 y que PA1 es el canal 2. Si se copiaba el anexo tal cual, el ADC iba a leer un pin donde no estaba conectado el potenciómetro, así que en el código se usó (2<<6) en lugar de (1<<6).

Secuencia de encendido (Fig. 3)

Se siguió el orden que indica el manual. Con BIC se limpió el bit 29 (DEEPPWD) de ADC2_CR y luego con ORR se encendió el bit 28 (ADVREGEN). Para darle tiempo al

regulador se hizo un retardo sencillo: se cargó 2000 en r2 y se le fue restando 1 con SUBS, mientras BNE regresa a la etiqueta adc_delay hasta que el resultado llega a cero. Con el reloj de 16 MHz con el que arranca el micro, esto tarda menos de un milisegundo, que es suficiente para que el regulador se estabilice.

Después se encendió ADEN (bit 0) y el programa se queda leyendo ADC2_ISR hasta que el bit ADRDY se pone en 1. TST hace un AND con la máscara del bit y BEQ vuelve a leer mientras el resultado sea cero.

Configuración de los LEDs (Fig. 4)

Los LEDs se conectaron en PA0, PA4, PA7 y PA2. Como estos pines arrancan en modo analógico (11), primero se limpiaron sus dos bits con BIC y luego se encendió solo el bit 2n con ORR para dejarlos en 01, que es salida. Es lo mismo que se hizo en C en la práctica de GPIO, solo que aquí cada operación es una instrucción. PA1 no se toca en esta subrutina, así que sigue en modo analógico.

Lectura del ADC (Fig. 5)

Para cada lectura se pone en 1 el bit ADSTART (bit 2 de ADC2_CR), que arranca la conversión. Luego se revisa en un ciclo la bandera EOC de ADC2_ISR hasta que vale 1 y se lee ADC2_DR. El resultado se deja en r0 para que lo use la siguiente subrutina. Como leer DR limpia la bandera EOC, no hace falta borrarla a mano antes de la siguiente conversión.

Cálculo de los rangos

Con 10 bits el valor máximo de la lectura es 1023. Para dividir el recorrido en cinco partes iguales se multiplicó ese valor por 1/5, 2/5, 3/5 y 4/5 y se redondeó al entero más cercano. Por ejemplo, para el primer umbral:

$$1023 \times \frac{1}{5} = 204.6 \approx 205$$

De la misma forma salen 409, 614 y 818. Para saber a qué voltaje corresponde cada umbral se usa la relación entre la lectura N y el voltaje de referencia:

$$V = N \times \frac{3.3\ V}{1023}$$

Con esto quedan los rangos de la siguiente tabla:

Recorrido	Lectura del ADC	Voltaje aprox.	LEDs encendidos
0 a 1/5	0 a 204	0 a 0.66 V	Ninguno
1/5 a 2/5	205 a 408	0.66 a 1.32 V	LED4
2/5 a 3/5	409 a 613	1.32 a 1.98 V	LED4 y LED3
3/5 a 4/5	614 a 817	1.98 a 2.64 V	LED4, LED3 y LED2
4/5 a 1	818 a 1023	2.64 a 3.3 V	Los cuatro

En el programa no hizo falta convertir la lectura a voltaje, porque comparar el número directamente da el mismo resultado y evita usar punto flotante. Como el potenciómetro es lineal, cada rango corresponde más o menos a un quinto del giro de la perilla.

Control de los LEDs (Fig. 6)

En led_write el registro r2 empieza en 0 y ahí se van juntando los bits de los LEDs que deben encenderse. La lectura se compara con cada umbral usando CMP, si es menor, BLO salta directo a led_out y ya no se agregan más LEDs. Si no salta, se agrega con ORR el bit del siguiente LED. Se usó BLO porque compara números sin signo, que es lo que entrega el ADC.

En led_out se lee el ODR, se limpian los bits de los cuatro LEDs con BIC usando la máscara 0x95, que tiene en 1 los bits 0, 2, 4 y 7, y después se combinan con r2 usando ORR. Así, con una sola escritura al ODR los LEDs quedan como deben y no se cambia ningún otro pin del puerto.

Modificación del paso 5 (Fig. 7)

Para el paso 5 se comentó la instrucción STR que escribe en GPIOA_ODR. El programa sigue configurando el ADC, haciendo conversiones y calculando qué LEDs deberían encenderse, pero ese resultado nunca llega a los pines. Al mover el potenciómetro de un extremo al otro los LEDs no cambiaron, lo que confirma que su estado depende únicamente de esa escritura.

Conclusiones:

- Al revisar el manual de la tarjeta se vio que PA2 y PA3 están conectados al ST-Link integrado, porque se usan para el puerto serial virtual (PA2 como TX y PA3 como RX). Aun así, PA2 se pudo usar sin problema para uno de los LEDs, ya que es la línea por donde el micro transmite y el ST-Link solo la escucha, así que al configurarla como salida no choca con nada. En cambio, PA3 es la línea por donde el ST-Link le envía datos al micro, y usarla como salida podría generar conflicto con esa señal. Esto dejó claro que, antes de elegir los pines, no basta con ver el pinout, también hay que revisar qué otras cosas de la tarjeta están conectadas a cada uno.

- No conviene copiar el código de ejemplo sin revisarlo. El anexo leía el canal 1 aunque configuraba PA1, y sin revisar el datasheet los LEDs no habrían respondido al potenciómetro. Revisar la tabla de pines antes de conectar ahorró bastante tiempo de depuración.

- Hacer la práctica en ensamblador ayudó a entender lo que hace el compilador con una línea como ADC2->CR |= (1<<2): en realidad son tres pasos, leer el registro, modificar el bit y volver a escribirlo. También quedó claro por qué el ADC necesita una secuencia de encendido y por qué hay que esperar sus banderas antes de usarlo.

Bibliografía:

[1] STMicroelectronics, "STM32G4 Nucleo-32 board (MB1430)", Manual de usuario UM2397, Rev. 2, sep. 2019. Accedido el 13 de octubre de 2026. [En línea]. Disponible: *https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf*

[2] STMicroelectronics, "STM32G4 Series advanced Arm-based 32-bit MCUs", Manual de referencia RM0440. Accedido el 13 de octubre de 2026. [En línea]. Disponible: *https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf*

[3] STMicroelectronics, "STM32G431x6 STM32G431x8 STM32G431xB", Hoja de datos DS12589, Rev. 6, oct. 2021. Accedido el 13 de octubre de 2026. [En línea]. Disponible: *https://www.st.com/resource/en/datasheet/stm32g431kb.pdf*

[4] W. Kester, "ADC Architectures II: Successive Approximation ADCs", Tutorial MT-021, Analog Devices. Accedido el 13 de octubre de 2026. [En línea]. Disponible: *https://www.analog.com/media/en/training-seminars/tutorials/MT-021.pdf*

[5] Arm Ltd., “Cortex-M4 Devices Generic User Guide”, ARM DUI 0553B, 2011. Accedido el 13 de octubre de 2026. [En línea]. Disponible: https://developer.arm.com/documentation/dui0553/latest/

[6] L. Llamas, "The Potentiometer: Your Hand Enters the Circuit". Luis Llamas. Accedido el 13 de octubre de 2026. [En línea]. Disponible: *https://www.luisllamas.es/en/potentiometers-types-connection-logarithmic-linear/*